ThermalBench

Developer Manual — a thermal-aware sustained-load inference benchmark for Android

A developer's guide to the architecture, the measurement method, and the day-to-day operation of ThermalBench.

Version	0.1.0
Generated	2026-10-26
Audience	Engineers building on, running, or reviewing ThermalBench
Source	GUIDE.md, and the code it describes

Contents

1 Why this tool exists
2 Three constraints that shape everything
3 Module topology
4 How a run actually works
5 The spec: what makes runs comparable
6 The precondition gate
7 Backends
8 Telemetry, and what it honestly cannot see
9 Analysis: from samples to a verdict
10 Running the headless harness
11 The operator console
12 Result documents and canonical serialization
13 Offline analysis and comparison
14 The leaderboard and consent
15 Building and verifying
16 Where to start, and where this goes

1. Why this tool exists

Every mobile inference benchmark reports peak throughput. Peak throughput is also the number a phone stops delivering after about ninety seconds. ThermalBench measures what happens after that: how throughput decays under sustained load, when the operating system begins throttling, and what fraction of peak the device can actually hold.

The gap this fills is documented. A March 2026 arXiv paper benchmarking a 4-bit Qwen 2.5 1.5B across mobile hardware found that thermal management, not peak compute, is the primary constraint on mobile platforms. The iPhone 16 Pro lost nearly half its throughput within two iterations. The Galaxy S24 Ultra hit an OS-enforced GPU frequency floor that terminated inference outright. The same paper notes that MLPerf Inference measures peak performance without tracking thermal throttling or sustained-workload behaviour.

Existing Android LLM applications report load time, memory use and a thermal state as a snapshot. Nobody has shipped the sustained degradation curve as tooling. That curve is what ThermalBench produces.

The headline number

The **retention ratio** is steady-state throughput divided by peak throughput. It answers the question a buyer, an app developer and a reviewer all actually have: of the performance this device advertises, how much of it survives a real workload? Every design decision in this manual exists to make that one number defensible.

> **What this tool is.** A measuring instrument that happens to have an app attached, not an app that happens to measure. When the two goals conflict, measurement integrity wins, and the code says so at each point where it does.

2. Three constraints that shape everything

Ordinary Clean Architecture explains most of this codebase. Three constraints explain the rest, and they are what separate a credible instrument from a phone benchmark app. Every unusual decision later in this manual traces back to one of them.

1. The observer effect is real and large

A Compose UI, live charts, logging and database writes all consume CPU and generate heat on the same die being measured. A benchmark whose own instrumentation perturbs the thermal envelope produces numbers nobody should trust.

So the primary execution path is **headless**: an instrumentation harness with no UI process at all. The Compose app is a convenience wrapper around a harness that runs without it. The result document records which path produced the numbers and whether live charts were on, so the two are never silently compared.

2. Preconditions are part of the measurement

A run starting from a warm device measures something different from one starting cold. Charging state, screen state, battery level and prior workload all change the answer. These are gated before a run starts and recorded in the result.

> A run that starts without satisfying its policy is not a slightly worse run. It is an **invalid** run, and the result document says so with `PRECONDITIONS_UNMET`. The tool never silently proceeds and never adjusts a number to compensate.

3. Time must be monotonic

Every duration comes from `SystemClock.elapsedRealtimeNanos()`. Wall-clock time appears exactly once per run, as a label for the document. A network time correction in the middle of a forty-minute run would silently corrupt a decay curve, and the corruption would be invisible in the output.

`elapsedRealtimeNanos` is chosen over `System.nanoTime()` deliberately: it keeps counting through deep sleep, which matters for runs that last tens of minutes with the screen off.

3. Module topology

Sixteen Gradle modules. Dependencies point inward only, and the direction is enforced by a convention plugin rather than by review.

```
core/domain          pure Kotlin. Entities, ports, use cases. No Android.
core/analysis        decay fitting, steady-state detection, statistics.
core/schema          result document model + canonical serialization.

data/backend-api     backend registry, model files, generation timing
data/backend-litert       LiteRT-LM adapter (the reference backend)
data/backend-llamacpp     llama.cpp JNI adapter (GGUF)
data/backend-mediapipe    MediaPipe Tasks GenAI adapter
data/backend-executorch   ExecuTorch adapter (phase 2)
data/telemetry       thermal, power, frequency, memory sources
data/persistence     Room, DataStore, canonical artifact store
data/leaderboard     opt-in, consent-gated submission client
data/engine          composition root shared by harness and app

harness              HEADLESS runner (am instrument). Never sees :app.
app                  Compose MVVM operator console (optional)

tools/cli            JVM fleet orchestrator over ADB
tools/analysis-cli   offline re-analysis, comparison, plots
build-logic          convention plugins
```

The critical edge

`harness` must never depend on `app`. If it does, the headless path is lost and with it the credibility of every number the tool produces. This is enforced mechanically by `DependencyRulesPlugin`, which fails the build at configuration time.

Adding that dependency produces exactly this, and no APK:

```
Dependency rule violation — arrows must point inward only:
  :harness (implementation) -> :app
```

The full rule set:

Layer	May depend on
core/…	core/… only
data/…	core/…, and data/backend-api
data/engine	core/…, and any data/… module
harness	core/…, data/… — never app
app	core/…, data/…, harness
tools/…	core/…, tools/… (JVM only, never Android)

Why data/engine exists

The harness and the app both need the same run engine, assembled from the same backend registry, telemetry sources and repositories. Neither can depend on the other: the harness must never see the app, and both are Android application modules, which Gradle cannot make one depend on the other anyway.

So the dependency-injection wiring lives in one library that both consume. It is the only module permitted to reach sideways across the data layer, and the convention plugin allows that for this module alone. The alternative, duplicating the wiring in two application modules, is exactly the kind of drift the plugin exists to prevent.

Why core/analysis is pure Kotlin

The same decay-fitting code powers the live on-device chart and the offline cross-device report. Two implementations of the same statistics is how a discrepancy ends up in a published paper. There is one, and `tools/analysis-cli` exists partly to prove it by re-deriving on a workstation the numbers a phone reported.

4. How a run actually works

A benchmark run moves through five phases. Understanding them is enough to read almost any part of the codebase.

1. **Validate the spec.** `ValidateSpecUseCase` checks the spec against the resolved backend's capabilities and rejects anything that cannot produce comparable results — before any device time is spent.
2. **Wait at the gate.** `AwaitPreconditionsUseCase` emits a state stream until every condition has held continuously for the stabilization window, or the maximum wait elapses.
3. **Load the model.** The backend adapter verifies the file against the digest in the spec, then loads it. Load time is recorded.
4. **Warm up, then measure.** Warmup iterations are discarded from the analysis and still written to the document. Measured iterations follow back-to-back by default.
5. **Analyze and write.** Samples become a `DegradationProfile`, and the whole run becomes a canonical JSON document plus a CSV sidecar.

The sampling loop

Telemetry sampling runs on its own dedicated thread and its own coroutine, entirely independent of the inference loop. If they shared a dispatcher, a long generation call could delay a thermal sample and leave a gap in precisely the series being measured.

Both write into one bounded channel consumed by a single collector that owns ordering and assembles the result. That is the only place run state is mutated, so there is no lock and no interleaving to reason about.



Two producers, one consumer. Neither producer can starve the other, and the consumer is the only writer of run state.

Two independent thermal streams

Polled samples give a regular time series suitable for plotting. Listener transitions give exact throttle-onset timestamps that polling would blur. Both are recorded, and the analysis prefers listener events for onset timing, falling back to iteration boundaries only when the platform delivered none.

5. The spec: what makes runs comparable

A `BenchmarkSpec` is the complete, immutable description of a run. Anything that could change the numbers belongs in it.

Its identity is the SHA-256 of its own canonical serialization. Two people who write the same spec get the same identifier, so results are comparable by construction rather than by convention. The model path and the run label are excluded from that hash, because they differ between devices without changing what is measured.

Field	Purpose
model	Name, format, SHA-256 digest, size. The digest is the identity; the path is a hint.
backend	Which adapter, which delegate (CPU / GPU / NPU), and any backend-specific options.
workload	Fixed prompt, its hash, prompt length, output length, and the stop condition.
protocol	Warmup and measured iteration counts, inter-iteration delay, abort policy, time limit.
preconditions	The gate the device must pass before the run may start.
seed	Recorded even where a backend decodes greedily and ignores it.
toolVersion	The version of ThermalBench that produced the run.

The stop condition is a token count, never end-of-sequence

> If one backend stops on end-of-sequence and another on a fixed token count, their tokens-per-second figures are not comparable. Output length is fixed across all backends and recorded. A spec whose backend cannot honour a token-count stop is **rejected** by the validator before any device time is spent.

What the validator catches

- A stop count that disagrees with the declared output length.
- A backend that is not registered, or is unavailable on this device, with the reason attached.
- A model format the chosen backend cannot load.
- A delegate the chosen backend does not offer.
- A backend that cannot enforce a token-count stop, and so cannot produce comparable rates.
- A stabilization window longer than the maximum wait, which could never be satisfied.

Too few measured iterations is a warning rather than a rejection. A short smoke run is a legitimate thing to want; the analysis marks the result `INSUFFICIENT_DATA` rather than the validator refusing to run it.

Where model files have to live

ThermalBench ships no models. It measures whatever you point it at, which is why the model's digest is part of the run identity.

> Both the app and the harness target a scoped-storage SDK, so neither can open an arbitrary path under `/sdcard` however the manifest is written. Models, specs and results all live in the relevant app's own external files directory. It needs no permission and adb can still reach it.

Consumer	Directory
Headless harness	`/sdcard/Android/data/com.thermalbench.harness/files/models`
App, debug build	`/sdcard/Android/data/com.thermalbench.debug/files/models`

The app lists whatever sits in that directory and lets you tap one, which sets the format from the file extension rather than leaving it to be typed. The extension is the only thing identifying a format here, so a file with an unrecognised one is listed as ignored rather than guessed at. The fleet tool places models with:

```
thermalbench push-model --model gemma-3n-e2b-it-q4.litertlm
```

Generating a spec

```
./gradlew :tools:cli:installDist
./tools/cli/build/install/thermalbench/bin/thermalbench spec > my-spec.json
```

The template comes out with a fixed reference prompt, its hash already computed, and the spec identity already derived. Edit the model path and the backend, then re-derive the identity by loading and re-saving it through the codec.

6. The precondition gate

The gate is what makes two runs comparable at all. It is a policy plus a stabilization window, and it emits a live state stream so an operator can see what is blocking a run.

Condition	Default	Why
Thermal status	NONE	A warm start measures a different thing than a cold one.
Thermal headroom	≥ 0.5 margin	Continuous 0..1 signal, far richer than the coarse status enum.
Unplugged	required	Charging is a heat source in its own right.
Battery	≥ 50%	A low battery triggers its own independent throttling.
Screen	OFF	Must be fixed rather than incidental; the panel is a heat source.
Stabilization	30 s	Conditions must hold continuously, not merely be true once.
Maximum wait	15 min	Beyond this the gate times out rather than waiting forever.

> **The window restarts on any failure.** Holding for twenty seconds, briefly charging, then holding again is not a stable device. Treating it as one would put a warm start into a run that claims to be cold.

Following the cool-down with the screen off

The default policy wants the screen off, and turning it on to check progress would itself fail the screen condition and restart the window. In the app, the runner service therefore reports the cool-down in two ways that need no screen:

- **Notification.** The run's ongoing notification shows the margin against its target, for example `Cooling: margin 0.27 of 0.50`, with a progress bar toward the target. It is visible on the lock screen.
- **Spoken cues.** While the thermal conditions are unmet, the phone reads the margin aloud every 20 seconds: *"Current thermal margin is 0.27."* When only the thermal status is blocking, it says the status instead. Two one-off cues mark progress: *"halfway cooled"*, spoken only when the device started at least partly warm, and *"cool enough"*, which also says what is still blocking, or roughly when the run will start.

Every displayed and spoken margin goes through one formatter, `formatThermalMargin`. It shows two decimals and rounds down, so a margin is never presented as reaching a threshold it has not reached: 0.499 reads 0.49, not 0.50. The voice can still lag the screen by up to 20 seconds while the margin is moving.

Speech goes out as navigation guidance, like a satnav prompt. It plays with the ringer on silent and follows the media volume. The speech engine is started only when there is something to say and is shut down before the first iteration, so nothing of it is resident while the device is measured. The run itself is silent, apart from a closing *"run finished"*, with the retention percentage, or a notice that it stopped with an error. A device with no speech engine stays silent rather than failing the run.

Every sentence spoken is also written to logcat, so what the phone said can be checked afterwards:

```
adb logcat -s ThermalBench | grep said
```

The headless harness does not speak. It is driven from a computer, which already sees the same state as log lines.

Overriding the gate

Sometimes a run must happen on a device that cannot satisfy the policy. The harness accepts an explicit override:

```
-e ignorePreconditions true
```

The run then proceeds and the result is permanently marked `PRECONDITIONS_UNMET`, with the specific unmet conditions listed in the document. The leaderboard refuses such a result, the comparison tables exclude it by default, and the app shows a red banner. The override changes what the tool does, never what it reports.

Handling unavailable signals

Some vendors return NaN for thermal headroom. That becomes `null`, the headroom check is skipped, and the absence is recorded. The alternative, treating an unavailable reading as a failure, would make the tool unusable on those devices; treating it as a pass would be a quiet lie.

7. Backends

Every inference runtime sits behind the `InferenceBackend` port. Adapters live in `data/backend-*` and are assembled into a registry by dependency injection, so adding a module adds a backend.

Backend	Format	Delegates	Notes
`litert-lm`	`.litertlm`	CPU, GPU, NPU	The reference path. Its session API separates prefill from decode. NPU needs the vendor's libraries packaged; see below.
`mediapipe-genai`	`.task`	CPU, GPU	Also stops on end-of-sequence, so the window closes at the Nth token.
`llamacpp`	`GGUF`	CPU, GPU	The format the Google stack will not ship. Requires the NDK. GPU needs the Vulkan build and a GPU Vulkan can drive.
`qualcomm-genie`	`Genie bundle`	CPU, NPU	Qualcomm's own runtime (QAIRT). The compute unit is the bundle's. Needs the SDK.
`executorch`	`.pte`	—	Phase 2. Registered, reports itself unavailable.

Why LiteRT-LM is the reference

Its session API exposes `runPrefill` and `runDecode` separately, and `runDecode` produces exactly one token per call. That gives an unambiguous time-to-first-token, a real prefill rate, and a decode rate measured over a token count fixed by the spec rather than by the model's end-of-sequence behaviour. Nothing has to be inferred.

A fresh session is created for every iteration. Reusing one would let iteration two inherit iteration one's key-value cache, which means iteration two would be measuring a longer sequence — a different workload wearing the same label.

How MediaPipe is handled honestly

MediaPipe stops on end-of-sequence as well as on its token budget, so it cannot be told to produce exactly N tokens. Rather than let that quietly make its numbers incomparable, the adapter timestamps every streamed token and ends the measurement window at the Nth one. Tokens the runtime emits afterwards fall outside the window. A generation that stopped early is reported with the count it actually reached, which the analysis records as a shorter iteration rather than a faster one.

Unavailable backends stay visible

> A backend that cannot run on a device is still registered and still appears in the list, with the reason attached: a missing native library, an unsupported delegate, a phase-2 adapter. **A gap in coverage is a finding, not something to hide from a comparison table.** The same contract covers a device without an NPU and an adapter nobody has written yet.

Building the llama.cpp backend

The native half needs the NDK, CMake and a llama.cpp checkout, so it is off by default and the project builds on a machine that has none of them. Without it the Kotlin adapter still compiles and reports the backend unavailable.

```
git clone --depth 1 --branch b11879 \
  https://github.com/ggml-org/llama.cpp ~/src/llama.cpp
```

Then enable it in your user-level `~/.gradle/gradle.properties`, which overrides the project file, so a checkout path never lands in the repository. Use forward slashes, because a properties file treats a backslash as an escape:

```
thermalbench.nativeLlama=true
thermalbench.llamaCppDir=C:/Users/you/src/llama.cpp
```

The pinned release, `thermalbench.llamaCppTag`, stays in the project's own `gradle.properties`.

A directory that is not a llama.cpp checkout fails the build at configuration time with a message saying so, rather than as a CMake error three layers down.

Why the checkout is pinned

llama.cpp has no stable API and no stable performance. Functions are renamed between releases, and kernel and threading changes move tokens per second on the same device. A run's spec does not capture the runtime version, so an unrecorded upgrade would show up in a comparison as a difference between devices. That is the kind of hidden confound the rest of this manual exists to exclude.

So the project names one release, `thermalbench.llamaCppTag`, and the native build does two things with it:

- It stamps the checkout's actual version, from `git describe --tags --always --dirty`, into the library. `runtimeVersion()` returns it, so every result document names the llama.cpp release that produced it. A locally modified checkout reads `b11879-dirty`, so a modified runtime cannot pass as the release.
- It warns when the checkout is not the pinned tag. The build still completes, so a newer release can be tried, and the stamp records that it was.

Upgrading is two changes made together: check out the new tag, and move the pinned tag property to match it. Compile errors in `llamacpp_jni.cpp` after an upgrade mean the API moved; compare the calls against the new `include/llama.h`.

> Results from different llama.cpp versions are not comparable, even for the same spec on the same device. Check the runtime version in each document before putting two llamacpp runs in one table.

Always an optimised build

The native code is built with `CMAKE_BUILD_TYPE=Release` for every variant, debug included. AGP would otherwise compile it as Debug, and an unoptimised llama.cpp is roughly an order of magnitude slower. A benchmark built that way measures its own compiler flags rather than the device, and still produces a plausible-looking decay curve.

GPU offload with Vulkan

A llama.cpp build without a GPU backend offers only the CPU delegate. Asking it to offload layers is accepted and silently ignored, so a run labelled GPU would really run on the CPU. GPU offload comes from ggml's Vulkan backend, which is off by default because it needs more on the build machine:

- **From the NDK:** the C Vulkan headers, the Vulkan loader and the `glslc` shader compiler. Nothing to install.
- **Two Khronos header checkouts:** Vulkan-Hpp (C++) and SPIR-V, which the NDK does not ship. Pin them to one Vulkan SDK release.
- **A compiler for the build machine itself.** llama.cpp compiles a small shader generator that runs during the build. On Windows this is Visual Studio Build Tools with the C++ workload; the build finds it with `vswhere`, so no developer shell is needed. On Linux and macOS, a gcc or clang on PATH is enough.

```
git clone --depth 1 --branch vulkan-sdk-1.4.363.0 \
  https://github.com/KhronosGroup/Vulkan-Headers ~/src/Vulkan-Headers
git clone --depth 1 --branch vulkan-sdk-1.4.363.0 \
  https://github.com/KhronosGroup/SPIRV-Headers ~/src/SPIRV-Headers
```

Then add three lines next to the llama.cpp ones in `~/.gradle/gradle.properties`:

```
thermalbench.llamaVulkan=true
thermalbench.vulkanHeadersDir=C:/Users/you/src/Vulkan-Headers
thermalbench.spirvHeadersDir=C:/Users/you/src/SPIRV-Headers
```

The first Vulkan build compiles several hundred shaders and takes noticeably longer than a CPU-only build; later builds are incremental. The APK gains `libggml-vulkan.so`, about 20 MB. A machine with its own host toolchain can pass it as the CMake argument `THERMALBENCH_HOST_TOOLCHAIN`.

Headers newer than the NDK's are safe on older devices. ggml loads every Vulkan entry point at run time, so the device's own driver decides what is available.

GPU is offered only when a GPU is found

A Vulkan build on a device whose driver exposes no usable GPU would fall back to the CPU without saying so. The adapter therefore asks ggml for a GPU device at startup and offers the GPU delegate only when one exists. Phone GPUs share system memory, so ggml lists them as integrated (IGPU); both kinds count. The decision is logged:

```
adb logcat -s ThermalBenchLlama
  GPU device: Vulkan0 (Mali-G78)
  load_tensors: offloaded 25/25 layers to GPU
```

llama.cpp's own log is routed to logcat under the same tag. Its load-time lines are the record of which device a run used and how many layers it offloaded, so check them before trusting a GPU result from a new device.

What to expect

GPU is not automatically faster. On a Pixel 6 (Mali-G78), qwen2.5-0.5B Q4_K_M decoded at about 16 tokens per second on the GPU against 8 on the CPU, but processed the prompt about four times more slowly, so each iteration took twice as long. Vulkan is the portable path. Qualcomm Adreno GPUs also have a dedicated OpenCL backend in llama.cpp, which this build does not yet include.

> A GPU and a CPU run of the same model have different spec identities, because the delegate is part of the spec. Compare them as two specs on one device, not as two runs of one spec.

GPU offload with OpenCL (Adreno)

llama.cpp also has an OpenCL backend written for Qualcomm Adreno GPUs. It needs only the Khronos OpenCL headers on the build machine, plus Python 3, which embeds the kernels. The OpenCL library itself is the GPU vendor's, on the device:

```
git clone --depth 1 --branch v2026.05.29 \
  https://github.com/KhronosGroup/OpenCL-Headers ~/src/OpenCL-Headers

# ~/.gradle/gradle.properties
thermalbench.llamaOpenCl=true
thermalbench.openClHeadersDir=C:/Users/you/src/OpenCL-Headers
```

Linking needs a `libOpenCL.so`, so the build generates a stand-in with every function the pinned `cl.h` declares. The app and harness exclude it from their APKs: packaged, it would shadow the vendor's library. The manifest declares `libOpenCL.so` with `uses-native-library`, which Android 12 and later require before an app may load a vendor library.

Every ggml backend is built as its own library and loaded by name at startup. A backend that cannot load is skipped and logged, so a phone without vendor OpenCL, such as a Pixel, still runs llama.cpp on the CPU and Vulkan. On a Snapdragon both OpenCL and Vulkan reach the same Adreno. The composer then offers a GPU API choice, recorded in the spec as the `gpuApi` option, and a GPU run is pinned to exactly one device. A spec that names no API gets OpenCL. The device that ran is recorded in the result's `computeDevice` either way.

NPU on Snapdragon (LiteRT-LM)

LiteRT-LM can run a model on the Qualcomm Hexagon NPU, but neither piece it needs is in its Maven package: a LiteRT dispatch library, and Qualcomm's QAIRT runtime libraries. Started without them, an NPU engine aborts the whole process natively. The adapter therefore offers NPU only when a `libLiteRtDispatch_*.so` is actually packaged.

1. Download the Qualcomm AI Runtime (QAIRT) SDK, version 2.42, from Qualcomm's software centre. It needs a Qualcomm account and accepting their licence, which is why it cannot be fetched by the build.
2. Build LiteRT's Qualcomm dispatch library with Bazel and NDK r28b or newer (Linux or WSL): `bazel build --config=android_arm64 @litert//litert/vendors/qualcomm/dispatch:dispatch_api_so`. It must come from the LiteRT release that matches the LiteRT-LM version in the version catalog; a mismatched pair fails at engine creation.
3. Collect, in one folder under `arm64-v8a/`: the dispatch library; `libQnnHtp.so`, `libQnnSystem.so` and `libQnnHtpPrepare.so` from `lib/aarch64-android/`; and the Hexagon stub and skel for the target chip, `libQnnHtpV75Stub.so` and `libQnnHtpV75Skel.so` for the Snapdragon 8 Gen 3 (SM8650).
4. Point `thermalbench.npuLibsDir` at the folder in `~/.gradle/gradle.properties` and rebuild.

With the property set, the libraries are packaged and the APK extracts its native libraries to disk, because the QNN runtime loads its DSP-side skel by directory path. The adapter points `ADSP_LIBRARY_PATH` at that directory before creating the engine. The manifest declares the vendor FastRPC library, `libcdsprpc.so`, which the runtime uses to reach the DSP. Without the property nothing changes: the APK carries no vendor code and NPU is not offered.

NPU models are compiled per chip. The catalog lists Gemma 3 1B for the SM8650 (`Gemma3-1B-IT_q4_ekv1280_sm8650.litertlm`), gated behind Google's Gemma licence like the other Gemma files. Google publishes NPU builds for SM8550, SM8650, SM8750 and SM8850 only. A file compiled for another chip fails when it loads.

> Older Snapdragons cannot run LLMs on the NPU by this route. The OnePlus 8's Snapdragon 865 (SM8250) has no published NPU models, so the honest result there is CPU against GPU.

Qualcomm Genie (QAIRT)

Genie is Qualcomm's own LLM runtime, part of the Qualcomm AI Runtime (QAIRT) SDK. It runs a model on the Hexagon NPU (the QnnHtp backend) or on Qualcomm's CPU path (QnnGenAiTransformer), and it is the most direct route to the NPU on a Snapdragon. The SDK is downloaded from Qualcomm under their licence and is never part of this repository. Point the build at an unpacked copy:

```
# ~/.gradle/gradle.properties
thermalbench.qairtDir=C:/PROJECTS/Android/2.44.0.260225
thermalbench.qairtHexagon=v75        # v73, v75, v79: one per NPU generation
```

The build then compiles a small JNI adapter against the Genie headers and packages only what running a prepared model needs: `libGenie.so`, the QNN system, HTP and GenAiTransformer libraries, and a stub and DSP-side skel per listed Hexagon version. The 85 MB `libQnnHtpPrepare.so` is left out: it compiles graphs on the device, and Genie bundles arrive compiled. The adapter opens `libGenie.so` at run time rather than linking it, so a build without the SDK, or a phone it cannot load on, reports the

backend unavailable with the reason.

The licence allows the libraries to ship inside an application, which is what an APK built this way is. It does not allow distributing them on their own, so they must not be committed. No account is involved at run time.

Model bundles

A Genie model is a folder: `genie_config.json`, a tokenizer and the model files. Push the folder into the models directory; the library lists it by folder name, and a spec's model path is its config. Genie resolves relative paths against the working directory, so the adapter makes every path in the config absolute before loading. It also makes sampling greedy with the run's seed, and caps generation at the spec's token count.

The compute unit is fixed when a bundle is built, so it is read from the config (QnnHtp is NPU, QnnGenAiTransformer is CPU) and preselected. A spec that asks a bundle for a different unit is refused at load, never quietly run elsewhere.

- **CPU bundles** are made on the PC with the SDK's `qnn-genai-transformer-composer` from a Hugging Face model, for the architectures it has configurations for: Llama 2/3.x, TinyLlama, Mistral, Gemma 2B, Phi-3 and others. TinyLlama 1.1B Chat converts in about 12 seconds to a 585 MB Z8-quantised bundle. On Windows the composer needs a small wrapper: the SDK's Python package always loads its Genie binding, whose Windows build does not load, although the composer never calls it.
- **NPU bundles** hold context binaries compiled for one Hexagon version. They are exported through Qualcomm AI Hub with the `qai-hub-models` package, which needs a free AI Hub account; Llama models also need Hugging Face access. The QAIRT version the bundle was compiled with should match the SDK the app was built with.

> Genie runs only on Snapdragon. On other silicon even its CPU path refuses to start (QNN error 1006, platform not supported; checked on a Pixel 6), so the backend reports itself unavailable there. Older Snapdragons are out too. Genie's CPU path uses ARMv8.6 int8 matrix instructions (smmla), and on a CPU without them it dies with SIGILL; a OnePlus 8T's Snapdragon 865 crashed this way. Its NPU path needs Hexagon v73 or newer (Snapdragon 8 Gen 2 onward). The adapter checks both before a run: it offers CPU only where /proc/cpuinfo lists i8mm, and NPU only on a known v73-or-newer SoC.

In the app, pick a `.zip` of the bundle folder with the file picker. The app unpacks it into a folder it owns, named after the zip, because a folder pushed with adb is one the app may not read. Unpacking streams one entry at a time into a hidden staging folder and moves the bundle into place only when it is complete and holds a `genie_config.json`. An entry whose path would leave the folder, or an archive that would unpack past the free space, is refused, and a cancelled or failed import leaves nothing behind.

On the harness, a bundle's folder must be created by the harness, like its other directories, before the files are pushed:

```
adb shell am instrument -w -e prepare true -e bundles tinyllama-cpu $RUNNER
adb push tinyllama-cpu/. $DIR/models/tinyllama-cpu/
```

The JNI layer

The JNI layer decodes one token per call for the same reason LiteRT-LM does: it is what allows an exact token-count stop. It also clears the context between iterations and decodes greedily, so sampling noise cannot masquerade as throughput noise.

8. Telemetry, and what it honestly cannot see

This is where most benchmark applications quietly fail. The table below is the realistic picture, not the optimistic one.

Source	API	Availability
Thermal status	`PowerManager.getCurrentThermalStatus()`	API 29+, universal. The primary signal.
Thermal headroom	`PowerManager.getThermalHeadroom()`	API 30+. Continuous 0..1. Some vendors return NaN.
Battery temperature	`ACTION_BATTERY_CHANGED`	Universal but coarse, and lags die temperature by minutes.
Charging state	`BatteryManager.EXTRA_PLUGGED`	Universal. Gated on, because charging is a heat source.
CPU frequency	`/sys/…/scaling_cur_freq`	Vendor dependent and increasingly restricted.
Hardware temps	`HardwarePropertiesManager`	Needs device-owner or system app. Not designed around.
Memory	`Debug.MemoryInfo`, `/proc/self/statm`	Universal.

Nothing is ever fabricated

> CPU frequency is returned only when **every** core is readable. A partial list never masquerades as the whole SoC, and an unreadable value is `null`, never a guess or a default. A result that says "CPU frequency was unreadable on this device" is more credible than one that quietly omits the field.

The same principle applies throughout. Battery temperature is recorded as corroboration and never treated as the primary thermal signal, because it lags the die by minutes. A thermal status the platform reports that this tool does not recognise becomes `UNKNOWN` and blocks a run rather than being optimistically read as cold.

9. Analysis: from samples to a verdict

`core/analysis` is pure Kotlin with no Android dependency, so the identical code runs on the device for the live chart and on a workstation for the paper.

Peak

The best rolling-median window, not the single best iteration. One lucky iteration is noise; a sustained window is a capability. Using a maximum here would inflate the denominator of the retention ratio and understate every device.

Steady state

A settled tail is required, defined by two conditions holding together: a coefficient of variation below 5%, and a normalized slope below 3% across the window. The steady-state value is the median of the settled tail.

If the last window is not itself steady, the run never stabilized and the result is `null`. Reporting the final value of a still-declining curve as a floor would be a fabrication, and a plausible one.

Decay fit

Levenberg-Marquardt against $a + b \cdot \exp(-t/\tau)$, giving an asymptote, an amplitude, a time constant and an R^2. A fit that does not converge, or a series too short or too degenerate to fit, is reported as absent rather than replaced by a straight line dressed up as a decay.

The validity verdict

Verdict	Meaning
`VALID`	Preconditions were met and enough iterations completed.
`PRECONDITIONS_UNMET`	The gate was not satisfied. Not comparable with other runs.
`INSUFFICIENT_DATA`	Too few measured iterations for sustained-load analysis.

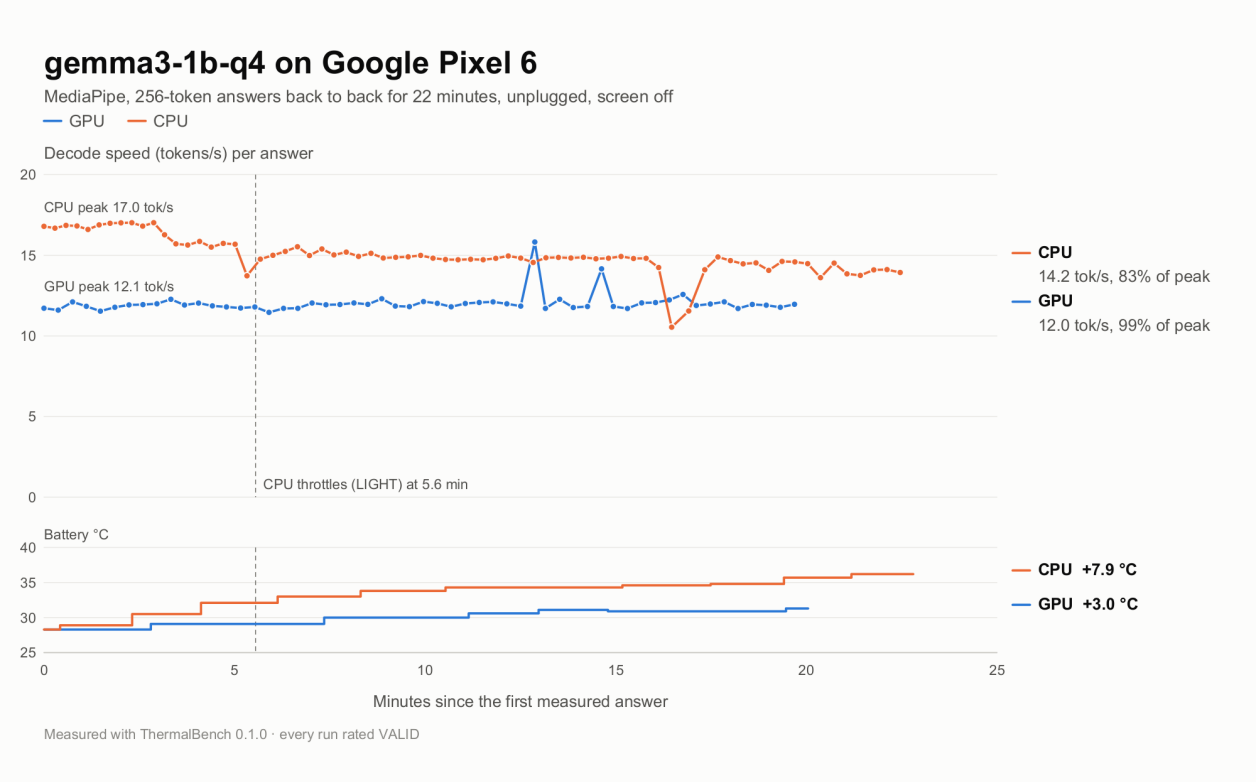
> **A run cut short by the operating system's thermal floor stays VALID.** That is not a failed measurement, it is the finding — the same finding the Galaxy S24 Ultra produced in the paper that motivated this tool. Only a run that ended for an unrelated reason before reaching its protocol is marked as insufficient.

Everything the profile reports

- Peak and steady-state decode throughput, and the retention ratio between them.
- Time and iteration count to the first throttle event, and every throttle transition.
- The exponential decay fit, or an explicit absence.
- Time-to-first-token drift, in microseconds per iteration.
- Mean and minimum decode throughput across the measured iterations.
- Battery drain and battery temperature rise across the run.
- The termination reason and the validity verdict.

A worked example: Pixel 6

Two harness runs on a Pixel 6 show the rules above at work: the same model and prompt on the CPU and on the GPU, each run unplugged with the screen off, both VALID.



Gemma 3 1B via MediaPipe on a Pixel 6 (Tensor G1), 256-token answers. The GPU's peak is 12.1 tok/s, its best sustained window, not the single 15.8 tok/s iteration at minute 13. The CPU run was still declining when it ended, so the analyzer reports no steady state and therefore no retention ratio, rather than treating the last value as a floor. Line-end labels give the mean of the last ten iterations as a share of peak (83% CPU, 99% GPU), reported beside the retention ratio, never instead of it. Drawn with `thermalbench-analyze plot`.

Read together, the two runs are the reason this tool exists. The CPU leads by about 40% at the start, throttles at 5.6 minutes and raises the battery 7.9 °C; the GPU holds 99% of its peak and raises it 3.0 °C. A benchmark that stops after a minute reports the first fact and misses the second.

10. Running the headless harness

This is the path that produces publishable numbers. It is a single self-instrumenting APK with no launcher activity: `am instrument` starts it, the benchmark runs, a document is written, the process exits. No UI process, no rendering load, and no second test process to perturb the measurement.

Install and run

```
./gradlew :harness:installDebug

RUNNER=com.thermalbench.harness/androidx.test.runner.AndroidJUnitRunner
DIR=/sdcard/Android/data/com.thermalbench.harness/files

# Let the harness create specs/, models/ and results/ itself.
adb shell am instrument -w -e prepare true $RUNNER

adb push my-model.litertlm $DIR/models/
adb push my-spec.json $DIR/specs/

adb shell am instrument -w \
  -e specPath $DIR/specs/my-spec.json \
  -e outputDir $DIR/results \
  $RUNNER

adb pull $DIR/results/
```

> Never create those directories with `adb shell mkdir`. A directory adb makes there belongs to shell: the harness cannot read a spec pushed into it, and adb cannot pull results the harness writes into it. `chmod` does not repair it. The harness checks who owns each directory before any device time is spent, and names the commands that fix it.

Phones that freeze background apps

Some vendors add their own power manager on top of Android's. OnePlus, Oppo and Realme phones run one (HANS) that freezes background processes about 25 seconds after the screen changes state. A frozen harness does not fail: the run simply stops mid-generation and never finishes. Adding the harness to the battery-optimisation whitelist does not prevent it.

The harness protects itself by holding a foreground service for the whole run, which these managers leave alone. Android grants that service only while the app is not background-restricted, and on these phones an app is restricted unless it has been explicitly allowed. Allow it once per phone, and again after uninstalling the harness:

```
adb shell cmd appops set com.thermalbench.harness RUN_ANY_IN_BACKGROUND allow
```

When the setting is missing, the harness logs a warning that gives this command at the start of the run (`adb logcat -s ThermalBench`). The app needs no such step: it starts its foreground service from the screen you tap.

Instrumentation arguments

Argument	Meaning
`specPath`	Path to a spec document on the device. Required unless preparing.
`prepare`	Only create the harness's directories and report where they are.
`bundles`	With prepare: model bundle folders to create under models/, comma-separated.
`outputDir`	Where result documents are written. Defaults to the app's external files directory.
`iterations`	Overrides the spec's measured iteration count. For smoke tests only.
`ignorePreconditions`	Runs once the gate times out instead of failing. The result is marked PRECONDITIONS_UNMET. The full wait still happens first.
`preconditionTimeoutSeconds`	Overrides the spec's maximum wait at the gate. With ignorePreconditions, a short value makes a quick smoke test.

The harness derives the spec identity from the spec it actually runs, not from the `specId` written in the file. A spec edited by hand, or one whose iteration count was overridden, is recorded under its own identity. The harness logs a warning when the file's identity was stale.

Reading the result

The harness reports its headline numbers back through the instrumentation status bundle, so a fleet script can read them without scraping logcat:

```
INSTRUMENTATION_STATUS: thermalbench.validity=VALID
INSTRUMENTATION_STATUS: thermalbench.retentionRatio=0.4713…
INSTRUMENTATION_STATUS: thermalbench.peakDecodeTokensPerSecond=16.6…
INSTRUMENTATION_STATUS: thermalbench.measuredIterations=50
INSTRUMENTATION_STATUS: thermalbench.document=/sdcard/…/result.json
```

Across a fleet

```
./gradlew :tools:cli:installDist
cd tools/cli/build/install/thermalbench/bin

./thermalbench devices               # what is attached, and is the harness on it
./thermalbench spec > my-spec.json   # a template to edit
./thermalbench run --spec my-spec.json --screen-off
./thermalbench pull --out results/
```

`--screen-off` matters more than it looks. Screen state is a precondition, and setting it from the orchestrator means every device in the fleet runs under the same one instead of whatever state each happened to be left in.

11. The operator console

The Compose app is optional by design. It exists for interactive use, and the numbers it displays come from the same engine the harness runs.

Screens

Screen	Purpose
Home	Run history with retention ratio and validity per run.
Spec composer	Pick model, backend, delegate and protocol; validate before committing.
Precondition gate	Live view of what is blocking the run and how close the device is.
Live run	Decay chart, thermal strip, throttle state, abort control.
Result detail	The full degradation profile, validity verdict, and export.
Compare	Backends on this device, or this device against imported results.
Leaderboard	Community results, with an explicit consent flow before any upload.

The run is a foreground service, not WorkManager

> Runs last tens of minutes and must survive Doze and screen-off. WorkManager may defer or reschedule work based on system conditions, which is **precisely the confound this tool exists to measure**. The runner is a foreground service of type `specialUse` with that justification in the manifest.

The service is handed only a spec identifier and re-resolves everything else from persistence, so restarting it is idempotent. It is deliberately `START_NOT_STICKY`: a run the system killed has a hole in its thermal history, and silently restarting it would produce a document claiming to be one continuous measurement when it is two.

A wake lock is held for the whole run and released in a `finally` block. It affects power draw, so the result document records that it was held.

The live chart caveat

The live chart is the single largest source of observer effect in the app path. Three things reduce it: chart recomposition is throttled to roughly 1 Hz, the series are held in a bounded ring buffer so a long run cannot grow memory without limit, and a minimal-UI toggle suppresses the charts entirely — available while a run is in progress.

> **A measurement worth publishing.** Run the same spec twice on one device, once charted and once headless, and publish the delta. It is a small credible result in its own right, and it inoculates the project against the most obvious reviewer objection. The result document records `executionPath` and `liveChartEnabled` precisely so that comparison is possible after the fact.

12. Result documents and canonical serialization

A finished run is stored twice on purpose. The canonical JSON document is the shareable artifact and the authoritative copy. The database holds a summary index, so the history list does not parse every document, and the in-flight samples that make a crashed run recoverable.

What canonical means here

- Object members sorted by key, so ordering is never a source of difference.
- No insignificant whitespace.
- Floating-point numbers written as the shortest decimal that reparses to the identical double.
- Non-finite numbers become null, since JSON cannot express them.

Encoding is **lossless**: decoding a document and re-encoding it reproduces the same bytes, which is what makes a digest a stable identity.

> **A caught bug worth knowing about.** The encoder originally rounded to 15 significant digits. IEEE 754 doubles need up to 17, so a retention ratio of 14.2 ÷ 17.5 decoded to a *different number* than it encoded. The round-trip property test caught it. Rounding to a fixed digit count is not a cosmetic choice; it silently changes measurements as they pass through the file format.

Why get this right now

Canonical serialization is the one piece that is expensive to retrofit and free to get right up front, which is why it has property tests rather than examples. If ThermalBench later grows into a conformance toolkit, this code is reused verbatim while most other things change shape.

Crash safety

Samples are flushed to the database in batched transactions every few iterations rather than held in memory until the end. The batching serves two purposes at once: it keeps database I/O off the critical path of an iteration, where its own heat and latency would contaminate the measurement, and it bounds what a crash can destroy to less than one batch. A crash at iteration 47 keeps the 46 that completed.

Export

Every run writes a JSON document and a CSV sidecar, one row per iteration. CSV is the format reviewers will actually open.

13. Offline analysis and comparison

```
./gradlew :tools:analysis-cli:installDist
cd tools/analysis-cli/build/install/thermalbench-analyze/bin

./thermalbench-analyze summary   results/
./thermalbench-analyze compare   results/ --csv cross-device.csv
./thermalbench-analyze reanalyze results/
./thermalbench-analyze plot      results/ --out decay.png
```

Command	What it does
`summary`	Headline numbers per document: peak, steady state, retention, validity.
`compare`	Cross-device table sorted by retention ratio; optional CSV output.
`reanalyze`	Recomputes the profile from recorded samples and flags any disagreement.
`plot`	Speed and battery temperature over time as a PNG, each run labelled where its line ends. The same image the app's "Share chart image" produces.

reanalyze is a correctness tool

It recomputes each profile from the stored samples and reports where the new value differs from the one recorded at run time. A disagreement means the analysis code changed since the run, which is exactly when previously published numbers may need a correction. Surfacing that loudly is the point.

Guardrails in the comparison table

- Invalid runs are excluded by default and included only with `--include-invalid`, which shows their verdict.
- A table containing more than one spec identifier prints a warning, because only runs of the same spec are comparable.
- A document that fails to parse is reported by name rather than skipped in silence.
- Plots are anchored at zero on the y-axis, so a mild decline cannot be autoscaled into a cliff.

The figure in chapter 9 was produced by `plot` from the two Pixel 6 result files; `--width` and `--height` set its size (1600 × 1000 by default).

14. The leaderboard and consent

Submission is opt-in, per-run, and gated on explicit consent. The client is written so the entire set of fields that can leave the device is readable in one screen of code.

What is sent

- The analysis summary: peak, steady state, retention ratio, throttle timing, drift.
- The spec identity and digest, the model name and quantization, the backend and delegate.
- The SoC manufacturer and model, the Android release and SDK level.
- An anonymous per-install device identifier, derived by hashing and truncation.
- Whether the run was headless or charted, and whether preconditions were met.

What is never sent

- The prompt or any generated text.
- The device model name, unless that box is separately ticked.
- Per-iteration data, unless that box is separately ticked.
- Anything at all when consent has not been given.

> Each field is a separate choice rather than one blanket agreement, and only runs marked `VALID` may be submitted at all. A leaderboard populated with runs from warm, charging phones would be worse than no leaderboard.

15. Building and verifying

Toolchain

Component	Version	Why this one
AGP	8.7.3	Android Studio Ladybug refuses to open newer AGP versions.
Kotlin	2.2.21	Matches the Kotlin that LiteRT-LM 0.16.1 was compiled with.
compileSdk / targetSdk	36	Needs android.suppressUnsupportedCompileSdk=36 under AGP 8.7.
minSdk	29	The thermal status API's floor. Below it there is no primary signal.
LiteRT-LM	0.16.1	0.17.0 carries Kotlin 2.4 metadata a 2.2 compiler cannot read.
OkHttp	5.4.0 (strict)	5.5.0 requires compileSdk 37. Strict, so Ktor cannot raise it.

Common commands

```
./gradlew build                    # everything: compile, test, lint, R8, coverage
./gradlew verifyDependencyRules    # the layering guarantee, on its own
./gradlew apiCheck                 # the published core API surface
./gradlew apiDump                  # accept an intentional API change
./gradlew detekt                   # static analysis
./gradlew koverHtmlReport          # coverage
```

Two environment notes that cost real time

- **Lint version.** The lint checks bundled inside androidx libraries are built against a newer Kotlin Analysis API than AGP 8.7.3's lint, and crash with "Found class Ka…, but interface was expected". `android.experimental.lint.version` pins a newer standalone lint so every check stays enabled instead of being switched off one by one.
- **Intermittent file lock.** On Windows a full build occasionally fails with a file lock on `bundleLibCompileToJarDebug`. It is environmental, not a code fault. Retry, and if it repeats run `./gradlew --stop` first.

APK size

Application modules package `arm64-v8a` only, which covers every device worth thermally profiling and avoids pushing three unused architectures to a fleet over ADB. Add `-Pthermalbench.includeX86=true` for an emulator smoke test. The ExecuTorch runtime is deliberately not linked while its adapter is a stub, because shipping a runtime nothing can call would add 8 MB to every APK for no benefit.

16. Where to start, and where this goes

Build order

The order below is deliberate: the project becomes useful at step 4 and publishable at step 6, long before it is complete.

1. `core/domain` and `core/schema`, with canonical serialization and property tests. No UI, no device.
2. `data/telemetry`, thermal first. Verify against a known-hot workload.
3. One backend — `backend-litert`.
4. `harness` producing a result document. **The project is useful here.**
5. `core/analysis` — decay fit, steady-state detection, retention ratio.
6. `tools/analysis-cli` across three or four devices. **Stop and write the preprint.**
7. `app`, with the charted-versus-headless delta measured and published.
8. Second and third backends, then `tools/cli` fleet orchestration.
9. Leaderboard last: it needs traffic to mean anything, and traffic follows the paper.

Growing into a conformance toolkit

If this later expands into an evidence and conformance toolkit, the seams are already cut.

Today	Becomes
`BenchmarkSpec`	A run manifest — add signing, no structural change.
`BenchmarkResult`	One section of an evidence bundle.
`DegradationProfile`	The sustained-performance-envelope evidence category.
`ResultRepository`	An evidence repository with signature verification added.
`Canonical serialization`	Reused verbatim. It is already the hard part.

The habit worth keeping

> Every place this codebase could have guessed, it records an absence instead: a null frequency reading, an absent decay fit, a steady state that never settled, a backend that will not load. That habit is the whole reason a number from this tool is worth quoting. Preserve it in anything added.